Notes On Examples

Sections.

1. Introductions.
2. Resources.
3. Notes on Examples.

Introductions.

The philosophy here is to introduce working examples of Web Assembly (wasm). Much of the material is taken from other works across the internet. We do not claim to be experts by any means on the subject.

It seems that the most useful part of *wasm* is to allow previously coded and tried material, in whatever language, to be executed within the context of a common browser, thus, widening audience capabilities and, perhaps, productivity.

Here is a description of what a wasm file is:

"WebAssembly (Wasm) modules are binary instruction sets designed for safe and efficient execution within modern web browsers and other runtimes. They are not JavaScript, but rather a distinct, low-level assembly-like language that serves as a compilation target for source code written in languages such as C++, Rust, Go, and Python. While JavaScript remains the primary scripting language of the web, Wasm operates alongside it to handle computationally intensive tasks that would otherwise be slow or unpredictable in a pure JavaScript environment. "

The bulk of the examples were patterned after the excellent documentation found at Mozilla′s Developer Network (MDN) site on Web Assembly.

The discussions presented within the examples are our interpretation of the processes stemming from the code and not from in depth study of the documentation, should you need more detail, please refer to the documentation listed under the resources section.

Resources.

- Web Assembly: https://webassembly.org/ ; The official site.
- MDN Web Assembly: https://developer.mozilla.org/en-US/docs/WebAssembly ; Mozillas documentation on Web Assembly.
- WebAssembly Specification: https://webassembly.github.io/spec/core/index.html
- Emscripten, A compiler tool chain to WASM: https://emscripten.org/index.html ; The official site.
  - This site provides a complete compiler toolchain to WebAssembly, using LLVM
  - installation of the emsdk
  - Using the emcc compiler to compile C/C++ files to JavaScript
- C/C++ to Wasm with Emscripten https://www.webassembly-wasm.com/compilation-pipelines-toolchain-setup/cc-to-wasm-with-emscripten/
- WebAssembly Scripting IDE https://www.assemblyscript.org/
- WAT Basics https://www.webassembly-wasm.com/webassembly-core-concepts-browser-runtime/webassembly-text-format-wat-basics/
  - install WABT (WebAssembly Binary Toolkit)
  - provides the following tools: wat2wasm, wasm2wat, and wasm-validate
  - install wasmtime to run wasm files with resources from WASI (WebAssembly System Interface)
- Introductory Site for WAT https://coderundebug.com/learn/wat/introduction/

- WAT Reference Manual: https://coderundebug.com/learn/wat-reference/modules/#variables
- Debug WASM in Chrome Developer Tools: https://developer.chrome.com/docs/devtools/wasm

Notes On Examples.

hello.c

```
#include <stdio.h>

int main()
{
   printf("Hello world!\n");
   return 0;
}
```

This traditional example introduces the basic processes that occur when compiling a C language programme to be executable by Javascript through the magic of Web Assembly.

We will avail ourselves of the Emsripten compiler ′emcc′ to compile hello.c to a wasm module which we will then load and execute within a browser environment,

```
emcc hello.c -o hello.html
```

This will generate three files:

- an object file: hello.wasm
- a javascript file: hello.js
- an HTML file: hello.html

The HTML file is the ″main″ process which starts several other sub-processes,

- by default, it codes a canvas, imitating somewhat a typical console in order to display the ″Hello World!″ string;
- it instantiates the wasm module;
- it loads the hello.js file.

Note that we used an i/o instruction, ″printf″ in our C code: the code in the hello.js file handles these basic i/o function calls, among other tasks.

helloV2.c

```
#include <emscripten.h>

int main()
{
  // Javascript functions run in C
   emscripten_run_script("alert('I have been called from C!')");
   emscripten_run_script("squareIt(12)");
   return 0;
}
```

The above code works only if you have installed the Emscripten sdk, 'emsdk'; see resources above, 'Emscripten, A compiler tool chain to WASM'.

The function *emscripten_run_script*, from *emscripten.h*, can be used to run any accessible Javascript function either from its library or a custom coded function.

fibonacci.c

```
int fib(int n)
{
   if(n == 0 || n == 1)
      return 1;
   else
      return fib(n - 1) + fib(n - 2);
}
```

This is pure C code without any includes. It defines a function that will be called within the Javascript code of a custom HTML file.

We compile the code with,

```
emcc fibonacci.c -o fibonacci.js -s EXPORTED_FUNCTIONS=_fib
-s EXPORTED_RUNTIME_METHODS=ccall,cwrap
```

This will not generate an HTML file for us as we will be coding a custom HTML file.

HTML

```
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="utf-8"/>
        <title>Fibonacci</title>
   <!-- load our javascript -->
   <script src="fibonacci.js"> </script>
   <script>
     /*
       we add another inline script file here to wrap our fib function
       into a method that we can easily call from our script.
       First we must wait for the runtime to be completely ready.
       For that we use the module, Module.onRuntimeInitialized
     */

     Module.onRuntimeInitialized = () =>
     {
        init();
     };

     function init()
     {
     /*
```

```
        var js_wrapped_fib = Module.cwrap("fib", "number", ["number"]);
          // use the variable to call the function
          console.log("The result of fib(5) is:", js_wrapped_fib(5));
        */

          var result = Module.ccall('fib', // name of C function
          'number', // return type
          ['number'], // argument types
          [5]); // arguments
          console.log("The result of fib(5) is:", result);

       }
     </script
   </head>
      <body>
           <p>Open the console to see the result! (f12)</p>
      </body>
</html>
```

Looking at the emcc compile command, we tell the compiler to make the C function visible to Javascript: *'-s EXPORTED_FUNCTIONS=_fib'*. Note that the underscore preceeding the function name is required.

The compiler will not expose functions that don't seem to be used at the time of compile, therefore, we must insist that functions we intend to use are kept visible. We do this with the compile option:

-s EXPORTED_RUNTIME_METHODS=ccall,cwrap

Since we intend to use functions within the binary wasm module, we must first instantiate it within our Javascript code. This occurs when loading the autogenerated *fibonacci.js* code; however, as Browser processing is asynchronous, we must wait for the wasm module to be fully loaded and ready before we can use it. This is accomplished by setting a callback function to the Module's *onRuntimeInitialized* property,

```
Module.onRuntimeInitialized = () =>
 {
     init();
 };
```

The above code uses Javascript's arrow function: *'=>'* to call the init() function when the wasm module is ready. Note that *Module* here is the name of the object we interface with.

To make coding easier, we can wrap our C custom function, assigning the result to a variable. There are two methods for achieving this: *ccall* and *cwrap*, both functions belonging to the *Module* object, as shown above.

fibonacciV2.c

```
#include "emscripten/emscripten.h"

#ifdef __cplusplus
```

```
#define EXTERN extern "C"
#else
#define EXTERN
#endif

EXTERN EMSCRIPTEN_KEEPALIVE int fib(int n)
{
   if(n == 0 || n == 1)
      return 1;
   else
      return fib(n - 1) + fib(n - 2);
}
int main()
{
  return 0;
}
```

Compile with a simple,

emcc fibonacciV2.c -o fibonacciV2.wasm

This code is similar to *fibonacci.c* except that it directs the compiler to keep the *fib* function accessible via a directive from an Emscripten macro. This means that we need not declare externals within the compiler command itself, but rather, within our own code.

The emcc compiler emits only the *fibonacciV2.wasm* file. We create and handle the wasm file without benefit of either an autogenerated HTML or JS file. We create our own custom HTML file and javascript code.

HTML

```
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="utf-8"/>
        <title>Fibonacci2</title>
  <script>
      const wasiMock =
         {
           fd_write: (fd, iovs, iovs_len, nwritten) => 0,
           fd_close: (fd) => 0,
           fd_seek: (fd, offset_base, offset_high, whence, newoffset) => 0,
           fd_read: (fd, iovs, iovs_len, nread) => 0,
           environ_sizes_get: (environ_count, environ_buf_size) => 0,
           environ_get: (environ, environ_buf) => 0,
           proc_exit: (rval) => {},
           main: () => main() // may be required here if you specify a main in your c code
        };
        const importObject =
        {
```

```
            wasi_snapshot_preview1: wasiMock
          };

            WebAssembly.instantiateStreaming(fetch("fibonacciV2.wasm"), importObject).then(
            (obj) =>
              {
                // Call an exported function:
                var result = obj.instance.exports.fib(6);
                console.log("fib = " + result);
              }
            );
    </script
     </head>
     <body>
          <p>Open the console to see the result! (f12)</p>
     </body>
</html>
```

In the Javascript code we make use of the WEB Assembly function, *'instantiateStreaming'*,

```
WebAssembly.instantiateStreaming(fetch("fibonacciV2.wasm"), importObject).then(
          (obj) =>
            {
              // Call an exported function:
              var result = obj.instance.exports.fib(6);
              console.log("fib = " + result);
            }
      );
```

WebAssembly is an object built into every modern browser. The *instantiateStreaming* method both compiles and instantiates a streaming source object.

The first argument requires a *'result'* object or *'promise'* that points to the source of the wasm object in question. In this case we use the Javascript *'window.fetch'* method, whose argument is typically an URL that points to the wasm module. The *fetch* method returns a Promise that resolves to a Response object.

importObject, is an option of the *instantiateStreaming* method. In this case we must import (define) certain objects as the compiler command we used exposes them, and since we did not ask for the Javascript file to be emitted, it asumed we would be handling these objects ourselves.

Here is an official definition regarding the imported objects parameter,

"An object containing the values to be imported into the newly-created Instance, such as functions or WebAssembly.Memory objects. There must be one matching property for each declared import of the compiled module or else a WebAssembly.LinkError is thrown."

The objects that are exposed by the emcc compiler are listed above in the properties for *wasiMock.*

As you can see, these objects mostly deal with i/o.

We bind the *wasimock* object to the pre-defined property *wasi_snapshot_preview1*, which again was exposed by the *emcc* process and we define a new object, *importObject*, to house it. The name is arbitrary.

```
const importObject =
{
    wasi_snapshot_preview1: wasiMock
};
```

instantiateStreaming returns the following,

A Promise that resolves to a ResultObject which contains two fields:

- module: A WebAssembly.Module object representing the compiled WebAssembly module. This Module can be instantiated again or shared via postMessage().
- instance: A WebAssembly.Instance object that contains all the Exported WebAssembly functions.

instance has a property called *exports*, which returns an object containing, as its members, all the functions exported from the WebAssembly module, to allow them to be accessed and used by JavaScript.

We can now call the C function *fib* by using the *instance* object,

```
obj.instance.exports.fib(6);
```

dataXchV1

```
#include <string.h>     // for memset
#include "emscripten/emscripten.h"  // for EMSCRIPTEN_KEEPALIVE

#ifdef __cplusplus
#define EXTERN extern "C"
#else
#define EXTERN
#endif

#define BUFFER_SIZE 1024

static char buffer[BUFFER_SIZE];

EXTERN EMSCRIPTEN_KEEPALIVE char* get_buffer_ptr(void)
{
   return buffer;
}

EXTERN EMSCRIPTEN_KEEPALIVE int get_buffer_len(void)
{
  return (int)BUFFER_SIZE;
}

EXTERN EMSCRIPTEN_KEEPALIVE char getCharFromBuffer(int n)
{
  // arg n inidicates which index within the buffer
  if(n < 0 || n > sizeof buffer)
```

```
  {
    return 0xFF;
  }
  return buffer[n];
}

EXTERN EMSCRIPTEN_KEEPALIVE char setCharIntoBuffer(int n, char m)
{
 // arg n inidicates which index within the buffer
 // arg m is the character to change to

  if(n < 0 || n > sizeof buffer)
  {
    return m;
  }
  memset(buffer + n, m, 1);
  return m;
}

EXTERN EMSCRIPTEN_KEEPALIVE int main()
{
  // initialise buffer with all x's
  memset(buffer,'x',sizeof buffer);
  // change the first character in the buffer
  char a = 'a';
  int one = 1;
  memset(buffer + 1,a,one);

  return 0;
}
```

The above code demonstrates Javascript′s ability to interact with C′s code as a wasm module. It establishes a buffer and through getter/setters, allows Javascript to read/write to the buffer. The main function sets up some data in the buffer for contrast.

HTML

```
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="utf-8"/>
        <title>dataXchV1</title>

    <script>
     /*
       We cannot access a C array directly.
       We must do so through pointers to getter/setter functions coded within the programme.
     */
```

```
        ptrBuffer = null;
        intBfrLen = null;
        mainFunc = null;
        resultGetChar = null;
        resultSetChar = null;
        const bufferSize = 1024;

        WebAssembly.instantiateStreaming(fetch("z.wasm")).then(
        (obj) =>
          {
            // establish a link and execute the get_buffer function defined in c and compiled into z.wasm
            ptrBuffer = obj.instance.exports.get_buffer_ptr();    // buffer will be used between JS and C
as the data store
            // get a pointer to the buffer
            console.log("get_buffer_ptr = " + ptrBuffer);   // must use pointers to access non-global
storage areas
            // establish a link and execute the function that will provide us with c's buffer length
            intBfrLen = obj.instance.exports.get_buffer_len();
            console.log("get_buffer_len = " + intBfrLen);
            // we now have a shared storage area fo two way communication.

            // have c store some data for retrieval by js
            // establish a link and execute the main function in c
            mainFunc = obj.instance.exports.main();
            console.log("main was recognised");
            mainFunc;      // execute c's main function

            // have js retrieve the newly stored data
            // establish a link and execute the function that will fetch a character from c's buffer
            // note: c's buffer was initialsed to all x's in main(), executed above,
            // afterwhich the second character was changed to an 'a',  ascii code 97.
            resultGetChar = obj.instance.exports.getCharFromBuffer(1);  // get the second character in
the buffer
            console.log("second value in c's buffer is: " + resultGetChar);
            // convert the ascii code to a character
            console.log("second character in c's buffer is: " +  String.fromCharCode(resultGetChar));

            // have js place data in the common storage buffer
            // passing characters to c must be in UInt8Array form.
            uInt8CharacterString = new TextEncoder().encode("h");
            // establish a link and execute the function that write a character into c's buffer at position
buffer +2
            charSetBuffer = obj.instance.exports.setCharIntoBuffer(2, uInt8CharacterString[0]);
            console.log("character set:" + charSetBuffer );
            // to confirm the write, let's fetch it back
            resultGetChar = obj.instance.exports.getCharFromBuffer(2);
            console.log("third character in c's buffer is: " +  String.fromCharCode(resultGetChar));
            // this data is now available for c to work with.
```

```
            //
            // passing strings to wasm memory so that c can work with them.
            //
            const encoder = new TextEncoder();
            const decoder = new TextDecoder();
            const u8array = new Uint8Array(bufferSize);
            const strMessage = "The Quick Brown Fox";

            // Let us make use of a pre-existing buffer created by default in the wasm module.
            // an object called memory is autogenerated by the emcc compiler that has a property
            //    that points to a pre-established buffer called linear memory.
            //    It is exported as "memory".

            // establish a link and execute the function that allows us to access this buffer
            const wasmMemory = obj.instance.exports.memory;
            // get the address that buffer points to
            const memoryBuffer = wasmMemory.buffer;
            // show size of pre-established buffer
            console.log("memory buffer byte length: " + memoryBuffer.byteLength);
            // fetch all the bytes from buffer as UInt8 type bytes using the typed array UInt8Array
            const memoryBytes = new Uint8Array(memoryBuffer); // memoryBytes is a UIntArray type
            // we can now access any byte within the buffer: they are initially set to 0.
            console.log("First byte of WebAssembly memory:", memoryBytes[0]);
            // prepare a string for writing into the buffer
            const u8ArrayMsg = new Uint8Array(strMessage.length);
            console.log("message length:", strMessage.length);
            console.log("uint8array length:", u8ArrayMsg.length);
            encoder.encodeInto(strMessage, u8ArrayMsg);
            memoryBytes.set(u8ArrayMsg, 3);  // memoryBytes is a UInt8Array type and set is a
method from it's parent: TypedArray prototype.
            console.log("third byte of WebAssembly memory:", memoryBytes[3]);
            // translated to ascii
            console.log( "First byte of WebAssembly memory as ascii char: " +
String.fromCharCode(memoryBytes[3]));
            // get a block of bytes from the UInt8Array: e.g. our string
            console.log( "fetched string: " + decoder.decode(memoryBytes.subarray(3,
strMessage.length + 3)));

          }
        );

    </script>
     </head>
     <body>
          <p>Open the console to see the result! (f12)</p>
     </body>
</html>
```

We compiled the source code with optimization,

emcc dataXchV1.c -oz

This produces two files: z and *z.wasm*, we load the *z.wasm* file in our Javascript.

The side effect from ultimate optimization is that the extraeneous i/o files we had to take care of is no longer required.

The Javascript code also demonstrates string handling for wasm memory storage and retrieval.

The code is heavily annotated; therefore, I leave it to the reader.

dataXchV2.c

This code is exactly the same as *dataXchV1*, the only change is the compile command which causes a change in the Javascript code in the co-related HTML file: we are tasked with handling the exposed WASI i/o files. We compile without optimization,

emcc dataXchV1.c -o dataXchV2.wasm

Does not generate either an HTML nor a Javascript file, simply emits a wasm object file (module).

dataXchV2.c

```
// dataXchV3.c
// emcc dataXchV3.c -o dataXchV3.wasm

#include "emscripten/emscripten.h"

#ifdef __cplusplus
#define EXTERN extern "C"
#else
#define EXTERN
#endif

#define SIZE  10
int data[SIZE];

EXTERN EMSCRIPTEN_KEEPALIVE void add(int value)
{
  for (int i=0; i<SIZE; i++) {
    data[i] = data[i] + value;
  }
}

EXTERN EMSCRIPTEN_KEEPALIVE int* getData()
{
  return &data[0];
}

EXTERN EMSCRIPTEN_KEEPALIVE int getSize()
```

```
{
  return SIZE;
}

// token main function
EXTERN EMSCRIPTEN_KEEPALIVE int main()
{
   return 0;
}
```

HTML

```
<!DOCTYPE html>
  <head>
    <title>dataXchV3</title>
     <script>

      const wasiMock =
      {
        fd_write: (fd, iovs, iovs_len, nwritten) => 0,
        fd_close: (fd) => 0,
        fd_seek: (fd, offset_base, offset_high, whence, newoffset) => 0,
        fd_read: (fd, iovs, iovs_len, nread) => 0,
        environ_sizes_get: (environ_count, environ_buf_size) => 0,
        environ_get: (environ, environ_buf) => 0,
        proc_exit: (rval) => {},

      };
      const importObject =
      {
        wasi_snapshot_preview1: wasiMock,
        main: () => main() // may be required here if you specify a main in your c code
      };

        WebAssembly.instantiateStreaming(fetch("dataXchV3.wasm"), importObject).then(
        (wasmObj) =>
          {
            // data stores created in C are accessible in linear memory in wasm
            //    and can be accessed in JS by fetching the address, i.e., the offset in linear memory,
            //    of the data variable.
            // Linear memory can be accessed through the exported wasm object variable
"memory",
            //    and by accessing it's buffer property: memory.buffer.
            // I.E., While c has immediate access to linear memory by creating a storage space,
            //    Javascript has access through an exported property, memory.buffer.

            // Firstly, get the offset address of the data location in Linear Memory from C
            // This is accomplished by the underlying compiled unit returning a pointer to JS.
```

```
            var offset = wasmObj.instance.exports.getData();

            // Secondly, we retrieve the number of elements set aside for the storage array by C
            var numElements = wasmObj.instance.exports.getSize();

            // Finally, we can retrieve the data from C's array
            // We view the data as Uint32 type data bytes, 4 byte unsigned integer,
            //    as we intend to work with that type of data; therefore,
            //    we create a view on the storage area, i.e., Linear Memory
            // Typed Array args are (Array buffer, offset into the buffer, number of elements within the array buffer).
            var linearMemory = new Uint32Array(wasmObj.instance.exports.memory.buffer, offset, numElements);
            // We now have a UInt32Array, linearMemory, that views 40 bytes of Linear Memory as type Uint32,
            //    i.e., 4 byte integer data, and we have access to numElements (10) of them.
            // At this time we have no idea what data is stored there. We will shortly initialise this area of storage.
            // Note: array lengths should be shared in order to avoid out of bounds errors.

            // Now we overwrite the storage area with some data: values 0 through 9, here in Javascript
            for (let i=0;i<numElements;i++)
            {
              linearMemory[i] = i;
            }

            // At this point, we have data to work with in Linear Memory of the wasm module
            // Let's make changes to the storage area of Linear Memory created by JS above,
            //    but using the underlying compiled functions we created in c
            // Call the c code "add" function within the wasm module to manipulate the data placed in Linear Memory by the above code in javascript
            wasmObj.instance.exports.add(10);
            // confirm the changes by displaying them in the console, here in Javascript
            for (let i = 0; i < numElements; i++)
            {
              console.log(linearMemory[i]);
            }

          } // end of successful promise argument
        ); // end of then

      </script>

  </head>
  <body>
        <p>Open the console to see the result! (f12)</p>
    </body>>
</html>
```

The above code demosntrates that Javascript and C communicate through a common memory buffer, rergardless whether it is instantiated in C or Javascript.

The C code creates a storage area. WASM concentrates this storage area in a buffer which it calls Linear Memory. It does this to maintain the concept of ′sandboxing′.

The Javascript code wishing to access this storage area that C is working with within Linear Memory, has to acquire three pieces of information:

1. The offset within Linear Memory that the beginning of C′s storage area is located;
2. Number of elements set aside by C;
3. The address of Linear Memory in the wasm module.

Point 1, the offset is acquired with the following line of code,

```
var offset = wasmObj.instance.exports.getData();
```

Note that we are returning an address (pointer) to the location within Linear Memory that was set aside for C′s array.

```
int* getData()
{
  return &data[0];
}
```

The ″address″ returned is actually the offset into Linear Memory.

Point 2, the number of elements is acquired by calling the C function getSize,

```
var numElements = wasmObj.instance.exports.getSize();
```

Point 3, the address of the memory buffer (Uint32Array) is acquired using the following line of code,

```
var linearMemory = new Uint32Array(wasmObj.instance.exports.memory.buffer, offset, 10);
```

Of course, Javascript has no idea the type of data that is stored at that location in Linear Memory. It is up to the programmer to act accordingly.

In our example we treat the data as a Uint32 typed array of 10 elements.

We initialise the array using Javascript and update the data using C.

Finally, we retrieve the data and display the contents of storage as confirmation that Javascript did indeed interact with C′s data.

The Web Assembly Text Language (WAT)

We code *.wat* files and convert them to *.wasm* files using the tool *wat2wasm* mentioned under Resources, unless otherwise noted.

We code simple HTML files to instantiate the *.wasm* module.

First, we introduce some simple WAT grammar,

WAT Grammar

Be aware that wasm/wat is continuosly being updated and new features added.

All code in a WebAssembly module is either global data or functions. Global data is declared and defined outside of functions and must appear first in the module.

The code starts with the *module* entry in parenthesis. It encapsulates all other code.

```
(module

. . . globals . . .
. . . functions . . .

)
```

The definition of a function appears within parenthesis,

```
( func <signature> <locals> <body> )
```

for example,

```
(func func_name (param i32) (param i32) (result f64)
i32.const 42
 ...
)
```

The words *module*, *func*, *param* and *result* are keywords, as are others we will mention shortly.

The *func_name* is optional. Wasm itself uses indexes to identify functions and parameters, with the first function encountered given index 0 and the the first parameter given index 0 within a function. We can use this index number rather than names but it is simply easier to use names; however, when we disassemble a wasm file we will only see the indexes.

The grammar loosely follows s-expressions. An s-expression, also known as a sexpr or sexp, it is a way to represent a nested list of data. It stands for "symbolic expression".

Expressions, within parenthesis can be nested to any depth.

Single comments start with the double semi-colon ";;"

Multiple line comments start with "(;" and end with ";)"

When naming parameters, functions, local variables, loops, and other things within WAT, you have to start them with a $ character. After that it can only contain the following ASCII characters (and no spaces).

```
0 to 9
A to Z
a to z
followed by any of the following,
! # $ % & ' * + - / : < = > ? @ \ ^ _ ` | ~
```

All data including functions can be thought of as stored on a stack.

Functions can return more than 1 value as the values are simply pushed onto the stack for later retrieval. Of course, in reverse order.

All functions must ensure the stack is ′empty′ upon termination; however, if the function is declared with result(s), this is accounted for and will remain on the stack upon function exit. If the

function was called with arguments (popped onto the stack), these arguments must also be popped off the stack for function termination.

The call keyword is used to call another function. Any data the called function is to work on must be pushed onto the stack first, e.g.,

```
i32.const 1
i32.const 5
call $func_add
```

Here is an example of a wat function that may be called by Javascript,

```
(module
    (
        func $add (param $lhs i32) (param $rhs i32) (result i32)
        local.get $lhs
        local.get $rhs
        i32.add
    )
    (export "add" (func $add))     ;; must export functions we wish to call from JS
)
```

We will explain wat code when we get to the Examples, shortly. We use this example to demonstrate that S-Expressions can fold there functionailty by the use of sub_expressions, e.g., the same as above, folded -

```
;; example sexp folding: add3.wat
(module
  (
    func $add (param $lhs i32) (param $rhs i32) (result i32) (i32.add (local.get $lhs) (local.get $rhs))
  )
  (export "add" (func $add))     ;; must export functions we wish to call from JS
)
```

As noted above, although we can apply names to variables in our wat code,
the compiler strips these and replaces them with sequential numbers, as the following shows.

```
(module
  (type (;0;) (func (param i32 i32) (result i32)))
  (func (;0;) (type 0) (param i32 i32) (result i32)
    local.get 0
    local.get 1
    i32.add)
  (export "add" (func 0)))
```

It also "unflattens" or unfolds our code.
The instruction *local.get 0*, does not get the value 0 it gets the *contents* of the first parameter.
The words *module, func, type, param, result, local.n, type.add,* and *export* are all keywords.

Now, for examples.

add.wat

```
(module
    (func $add (param $lhs i32) (param $rhs i32) (result i32)
        local.get $lhs
        local.get $rhs
        i32.add
    )
    (export "add" (func $add))
)
```

The function name is called $add. It is defined with two parameters, $lhs and $rhs that are of type i32. The function is expected to return a value of type i32.

WAT supports the following data types:

i32	*32 bit unsigned integer 0 to 2^32*
i64	*64 bit unsigned integer 0 to 2^64*
f32	*32 bit floating point number*
f64	*64 bit floating point number*
v128	*128 bit vector containing either,* *2x 64 bit floats* *4x 32 bit floats* *2x 64 bit unsigned integer* *4x 32 bit unsigned integer* *8x 16 bit unsigned integer* *16x 8 bit unsigned integer*
funcref	*Reference to a function*

Numeric literals can be expressed in decimal or hexadecimal: e.g., 42, -42, 0x36.

Finally, we return to the function *add.wat*, which is converted to a wasm object file using the command,

```
wat2wasm add.wat
```

HTML

```
<!DOCTYPE html>
  <head>
    <title>Add</title>
```

```
    <script>
        const wasiMock =
        {
          fd_write: (fd, iovs, iovs_len, nwritten) => 0,
          fd_close: (fd) => 0,
          fd_seek: (fd, offset_base, offset_high, whence, newoffset) => 0,
          fd_read: (fd, iovs, iovs_len, nread) => 0,
          environ_sizes_get: (environ_count, environ_buf_size) => 0,
          environ_get: (environ, environ_buf) => 0,
          proc_exit: (rval) => {},
        };
        const importObject =
        {
          wasi_snapshot_preview1: wasiMock
        };
        // end of definition for exported objects

          WebAssembly.instantiateStreaming(fetch("add.wasm"), importObject).then(
          (obj) =>
            {
              console.log("The result of the add is: " + obj.instance.exports.add(1, 2)); // "3"
            } // end of successful promise argument
          ); // end of then
    </script>
  </head>
  <body>
     <p>Open the console to see the result! (f12)</p>
  </body>>
</html>
```

We start off discussions from the Javascript. Other than the boilerplate code, the instructions are straightforward.

The following line, after wasm instantiation, executes the *add* instruction,

```
console.log("The result of the add is: " + obj.instance.exports.add(1, 2)); // "3"
```

and passes two values to the *add* function: 1 and 2, expecting a return value of 3.

Moving then, to the wat instructions, we start with the function definition,

```
func $add (param $lhs i32) (param $rhs i32) (result i32)
```

The function expects that there are 2, *i32* type values passed to it.

The first task then is to grab those values passed to us from Javascript, which is done by the *get* method of the *local* object,

```
local.get $lhs
local.get $rhs
```

Local variables can be declared within a function, either implicitly in the function's parameter list or explicitly,

(local $my_var i32)

They must appear at the beginning of the function.

"WebAssembly local variables are stored within the function execution stack, which serves as an implicit, managed storage area for the virtual machine. This stack handles control flow, function arguments, return values, and local variables during the execution of a function frame. Values on this stack exist only for the duration of the active function and are discarded automatically when the function returns. The execution stack is completely managed by the runtime environment and is inaccessible to the WebAssembly program itself. "

The next instruction,

i32.add

pops 2, *i32* values off the stack, calculates and returns the sum which it *pushes* onto the stack.

The javascript call to the function,

obj.instance.exports.add(1, 2));

is returned the last value pushed onto the stack, as declared by the function.

funcAdd.wat

This example is similar to *add.wat* but demonstrate function calls within the wat code.

```
(module
    (func $getAnswer (result i32)
        i32.const 42
    )
    (func (export "getAnswerPlus1") (result i32)
        call $getAnswer
        i32.const 1
        i32.add
    )
)
```

There are two functions defined here. The second defined function calls the first. As they are both functions, we need code to start the ball rolling. This is accomplished through the Javascript code in the HTML file. We reprint the pertinent code here,

```
WebAssembly.instantiateStreaming(fetch("funcAdd.wasm"), importObject).then
(
    (obj) =>
```

```
    {
        console.log("The result of the add is: " + obj.instance.exports.getAnswerPlus1()); // "43"
    } // end of successful promise argument
); // end of then
```

It calls the exported function *getAnswerPlus1(),* without arguments; for the purposes of this example we hard code them.

getAnswerPlus1 calls *$getAnswer.*

$getAnswer executes the instruction,

```
i32.const 42
```

which pushes the i32 integer value onto the stack.

Upon return from *$getAnswer, getAnswerPlus1* pushes 1 onto the stack then adds the last two values onto the stack placing the sum back onto the stack.

Control returns to Javascript with the last value on the stack as defined by the called function *$getAnswerPlus1*,

```
(result i32)
```

importJSFunc.wat

Just as Javascript can import a function from a wasm module, a wasm module can import a Javascript function. In this example we demonstrate the importation of the *console.log* object/method.

```
(module
    (import "console" "log" (func $log (param i32)))
    (func (export "logIt")
        i32.const 13
        call $log
    )
)
```

The *import* instruction names the function to be imported from Javascript. Note that it is a two level declaration: *console* and *log*. It also defines a local function called *$log* that must conform to the imported function′s signature.

The second function, *logit*, is expected to be called by the Javascript code, which we will show shortly. This function pushes the 4 byte integer value 13 onto the stack then calls *$log;* which, in turn, executes the *console.log* function within Javascript. In other words, we can bind an external function to an internal function in this manner.

HTML:

```
<!DOCTYPE html>
  <head>
    <title>importJSFunc</title>
      <script>
```

```
        const wasiMock =
        {
          fd_write: (fd, iovs, iovs_len, nwritten) => 0,
          fd_close: (fd) => 0,
          fd_seek: (fd, offset_base, offset_high, whence, newoffset) => 0,
          fd_read: (fd, iovs, iovs_len, nread) => 0,
          environ_sizes_get: (environ_count, environ_buf_size) => 0,
          environ_get: (environ, environ_buf) => 0,
          proc_exit: (rval) => {},
          main: () => main() // may be required here if you specify a main in your c code
        };
        const importObject =
        {
          wasi_snapshot_preview1: wasiMock,
          console:
          {
            log(arg)
            {
              console.log(arg);
            },
          },
        };

          WebAssembly.instantiateStreaming(fetch("importJSFunc.wasm"), importObject).then(
          (obj) =>
            {
              obj.instance.exports.logIt();
            } // end of successful promise argument
          ); // end of then
    </script>
  </head>
  <body>
        <p>Open the console to see the result! (f12)</p>
    </body>>
</html>
```

You will note that the code above for the imported object for *console.log* defines *console* as an object having the property defining a function called *log*, which must have the corresponding bound Javascript function defined within it's body.

start.wat

This example is similar to *funcAdd.wat* except that it starts processing itself.

```
      (module
          ;; example using start
          ;; similar to importJSFunc.wat but is self running (start command)

            (import "console" "log" (func $log (param i32)))
```

```
        (func $main
            (local $var i32) ;; create a local variable named $var
            (local.set $var (i32.const 10)) ;; set $var to 10
            local.get $var ;; load $var onto the stack
            call $log ;; log the result
        )
        (start $main)
    )
```

'start' is a keyword used as the starting point in a module: it is similar to *main* in C.

The HTML instantiation code does not need to call the start function, it is self executing,

```
WebAssembly.instantiateStreaming(fetch("start.wasm"), importObject).then(
    (obj) =>
        {
        }
    );
```

Bin2Dec_FD_Write.wat

This next example does not use Javascript to launch the wasm module nor is it compiled using *emcc*, instead, we use the *wasmtime* tool which compiles and executes the resulting wasm code and provides WASI resources to the wasm module.

Bin2dec is a programme to convert a binary value to decimal ascii code sufficent to display on the console; however, getting a message to the user requires knowledge of the *fd_write* function and what needs to be stored where in wasm memory to satisfy its parameters.

We will also see new but common syntax, e.g., *if/end, global, loop/end* etc.

```
(module

   ;; this module, Bin2Dec_FD_Write.wat, requires WASI resources: i.e., fd_write
   ;; Run with: wasmtime Bin2Dec_FD_Write.wat

   ;; ***** DECLARATION SECTION *****
   (import "wasi_snapshot_preview1" "fd_write"
      (func $wasi_fd_write (param i32 i32 i32 i32) (result i32))
   )
   (memory (export "memory") 1)

   ;; ***** GLOBAL SECTION *****
   (global $intbin2decPtr (mut i32) (i32.const 64))        ;; ptr for the work area for the bin2dec function, which aggregates down memory
   (global $intbin2decReturnAddr (mut i32) (i32.const 0))  ;; the address returned by bin2dec to the start of the decimal string
   (global $intCharMsgAddr (mut i32) (i32.const 96))       ;;  start address of leading characters for the output message
```

```
(global $intCharMsgLen (mut i32) (i32.const 15))        ;; length of leading output string
(global $intOutPutArea (mut i32) (i32.const 32))        ;; start of staging area
(global $intFDIovec (mut i32) (i32.const 0))            ;; location of iovec for fd_write
(global $intFDDevice (mut i32) (i32.const 1))           ;; code for stdout
(global $intFDIovecs (mut i32) (i32.const 1))           ;; number of iovecs
(global $intFDBytesWritten (mut i32) (i32.const 8))     ;; number of bytes output by fd_write

;; ***** DATA SECTION *****
(data (i32.const 96) "The result is: ")

;; ***** GENERAL FUNCTIONS *****
(func $bin2dec (param $num i32) (param $ptr i32) (result i32)
   ;; Function to convert an unsigned 32-bit integer to a decimal ASCII string
   ;; Returns the memory offset pointing to the start of the string
   ;; $num: the number to convert from binary to ascii decimal characters
   ;; $ptr: Pointer to walk backwards in memory

   (local $digit i32)    ;; Holds the remainder (current decimal digit)

   ;; Start writing at the end of a safe buffer zone
   ;; An unsigned 32-bit integer has a max length of 10 digits

   ;; Handle the edge case where the input number is exactly 0
   local.get $num
   i32.eqz
   if
      ;; $num = 0
      (local.set $ptr (i32.sub (local.get $ptr) (i32.const 1)))
      (i32.store8 (local.get $ptr) (i32.const 48))              ;; ASCII '0'
      (return (local.get $ptr))
   end

   ;; Loop to extract base-10 digits
   loop $digit_loop

      ;; Extract the rightmost digit: digit = num % 10
      (local.set $digit (i32.rem_u (local.get $num) (i32.const 10)))
      ;; Convert the numeric digit to its ASCII character value (digit + 48)
      ;; and store it in linear memory at the pointer address
      (i32.store8
         (local.get $ptr)
         (i32.add (local.get $digit) (i32.const 48))
      )
      ;; Drop the extracted digit from the number: num = num / 10
      (local.set $num (i32.div_u (local.get $num) (i32.const 10)))
      ;; Move pointer backward by 1 byte
      (local.set $ptr (i32.sub (local.get $ptr) (i32.const 1)))
      ;; Continue looping if the remaining number is greater than 0
      (br_if $digit_loop (i32.gt_u (local.get $num) (i32.const 0)))
```

```
    end
    ;; Return the pointer where the decimal string starts
    (local.set $ptr (i32.add (local.get $ptr) (i32.const 1)))       ;; 1 was subtracted in anticipation of
looping: put it back
    (local.get $ptr)
  )

  ;;
  ;; ***** MAIN LOGIC *****
  ;;
  ;;  Define and export the entry point function
  (func $main (export "_start")
    ;;
    ;; init
    ;;
    (local $intResLen i32)    ;; length of characters in the result
    (local $intResOffset i32) ;; offset into the staging area for the result

    ;;
    ;; main process
    ;;
    i32.const 0x0F              ;; first arg for bin2dec: binary number to convert
    global.get $intbin2decPtr   ;; second arg for bin2dec: result work area
    call $bin2dec
    global.set $intbin2decReturnAddr    ;; save address of start of decimal string

    ;; build staging area for message to be output by fd_write
    ;; prepare for a memory copy
    global.get $intOutPutArea           ;; start of staging area
    global.get $intCharMsgAddr            ;; start of literal message data
    global.get $intCharMsgLen             ;; number of bytes to copy
    memory.copy

    ;; finish setting up staging area by appending result
    ;; get length of characters in result
    (local.set $intResLen (i32.sub (global.get $intbin2decPtr) (global.get
$intbin2decReturnAddr)))   ;; calculate length
    (local.set $intResLen (i32.add (local.get $intResLen) (i32.const 1)))   ;; add 1 for inclusivity
    ;; get location to append to end of string
    (local.set $intResOffset (i32.add (global.get $intOutPutArea) (global.get $intCharMsgLen)))   ;;
    local.get $intResOffset           ;; to address
    global.get $intbin2decReturnAddr    ;; from address
    local.get $intResLen              ;; length to copy
    memory.copy
    ;; add a line feed at end of msg
    local.get $intResOffset
    local.get $intResLen
    i32.add
    i32.const 0x0A
```

```
    i32.store8

    ;; setup fd_write
    ;;
    ;; build iovec
    ;;
    global.get $intFDIovec          ;; location of iovec
    global.get $intOutPutArea        ;; start address of msg
    i32.store                   ;;
    global.get $intFDIovec          ;; location of iovec
    i32.const 4
    i32.add                     ;; second part of iovec: memory offset for i32.store
    ;; calculate total length of msg and push it
    ;;
    local.get $intResLen
    global.get $intCharMsgLen
    i32.add
    i32.const 1                  ;; add 1 for the line feed control character
    i32.add
    i32.store

    ;; build wasi_fd_write
    global.get $intFDDevice          ;; device type: stdout
    global.get $intFDIovec          ;; address of iovec
    global.get $intFDIovecs          ;; number of iovecs
    global.get $intFDBytesWritten       ;; location to store number of bytes output

    call $wasi_fd_write
    ;; fd_write returns an error code (0 for success).
    ;; We use drop here to clean it off the stack.
    drop

  ) ;; end function
)  ;; end module
```

We start our discussions of the above code with the Global and Declaration sections.

The code imports the *fd-write* function whose signature is,

func $wasi_fd_write (param i32 i32 i32 i32) (result i32)

which requires 4, 4 byte integer arguments when called and returns a 4 byte integer value. The required argument and return values will be discussed shortly.

The code also exports the '*memory*' buffer (Linear Memory) established in the wasm module by the compiler. It exports it using the name 'memory' as well, for convenience.

The Global section creates variables to either hold addresses or hard coded values that are used within the functions defined later on in the code. They are similar to the concept of global variables in most application programming languages. Most of the global variables define offsets into Linear Memory (LM) where we place various values or iovecs to be passed as arguments.
(Aside. iovecs is the popular mnemonic for input output vectors, where a vector in computer science simply means a dynamic array of values.)

The Data section contains one or more data declarations, that is, it sets aside a defined amount of Linear Memory for textual or hard coded data:

(data (i32.const 96) "The result is: ")

A general format for this instruction is,

(data (i32.const offset_value) data)

where,

- *data*, is a keyword;
- the i32 constant defines the offset into Linear Memory where the data that follows will be inserted upon wasm module initialisation;
- the data is defined as a string surrounded in double quotes and can contain escaped hexadecimal values, e.g.,

  (data (i32.const 0) "\01\F2\AB\54")

It is defined here with text to be the start of our message to the user, to which we will be adding our binary to decimal result.

Skipping now to the Main Logic section of our code, we set up the stack prior to calling the *bin2dec* function.

Recall that when dealing with functions, the arguments must be pushed onto the stack, in parameter order, prior to calling the function. The result(s) are expected to be on the stack in reverse order, prior to exiting the function. The called function must retrieve its argument values from the stack in reverse order or, by name.

We start off by requesting the binary to decimal conversion using the *bin2dec* function. In order to do so, we must first place our arguments onto the stack, in order by the declared parameters.

The first parameter is the number to be converted,

i32.const 0x0F

which is 15 decimal

The second item pushed onto the stack is the offset into Linear Memory where the bin2dec function is to use as its work area.

We call the *bin2dec* function and upon return, we expect a 4 byte value remaining on the stack that represents the offset to the beginning of the resulting conversion as an ascii decimal string.

We will use this address to copy the result to the end of our message string destined to the user, that we initiated in the Global section. We expect the result string to have the ascii code for decimal 15: '3135'.

We will discuss the *bin2dec* function after we conclude with the Main Logic section.

The next steps in Main Logic involve,

1. creating a staging area for output to ther user's console
2. prepare the arguments for a call to the *fd-write* function

In these steps we attempt to avoid hard coding which inevitably adds complexity to the process, but, makes the module more flexible.

In step (1), we copy the initial message from the global variable to the staging area of Linear Memory.

To accomplish this we use the wasm method *memory.copy,* which requires the stack to have,

i32 The offset within memory to copy data to.

i32 The offset within memory to copy data from.

i32 The number of bytes to copy.

We achieve this by pushing the relevant data from our previously defined global variables.

```
global.get $intOutPutArea       ;; start of staging area
global.get $intCharMsgAddr      ;; start of literal message data
global.get $intCharMsgLen       ;; number of bytes to copy
```

We finish setting up the staging area by appending the result of the conversion. This requires us to, first, calculate the number of characters in the result, in order to fetch just those characters from *bin2dec*'s result area of LM (Linear Memory),

(local.set $intResLen (i32.sub (global.get $intbin2decPtr) (global.get $intbin2decReturnAddr)))

It may be easier to comprehend the above code if we process the above flattened s-expression right to left: subtract the addresse *$intbin2decReturnAddr* from *$intbin2decPtr* and store the result at *$intResLen.*

'$intResLen' is a local variable declared within the function,

(local $intResLen i32)

Local variables must be declared at the top of the body of the function.

'local' is a keyword and the general format is,

(local variable_name data-type)

The next step is to add 1 to this calculated length to include the last character, this gives us the length of the result string to copy.

(local.set $intResLen (i32.add (local.get $intResLen) (i32.const 1)))

'$intResLen' will hold our result string's length.

Now, we need to calculate the end of the initial string in order to have the offset as to where to append the result,

(local.set $intResOffset (i32.add (global.get $intOutPutArea) (global.get $intCharMsgLen)))

'$intResOffset' will hold the offset within LM to append our result. We take advantage of the built-in *memory.copy* method to do so, similar to above.

To avoid obfuscation at the console, we add a line feed character to the end of our message. To accomplish this we use the *i32.store8* method, which copies the first byte from the i32 stack value to the specified destination,

```
local.get $intResOffset
local.get $intResLen
i32.add
i32.const 0x0A
i32.store8
```

i32.store8 requires the stack have the following information,

i32 Memory offset location to copy the data to.

i32 The data to be copied into memory. Only the first 8 bits are copied.

In the code above *0x0A*, an ascii line feed character, is stored at the end of the result in the staging area.

We now are ready to set up and call *fd-write* to output our message to the console.
The *fd_write* WASI function requires 4 parameters in the following order,

1. a small positive 4 byte integer value corresponding to the file descriptor for the device type: 0=stdIn, 1=stdOut, 2=stdError;
2. the offset value within LM to the start address of the first of two 4 byte integers (vector) which describes the data to be copied. The first 4 bytes contains the offset in LM to the start of the buffer (data/message) to be written from, and the second contains the length of the buffer (data/message). There can be more than one vector. This process is known as scatter/gather type processing;
3. The number of vectors to be processed, starting at (2) above;
4. The location in LM to store a 4 byte integer containing the total number of bytes written by the *fd_write* function.

We start off by building the iovec (input output vector) at location $intFDIovec: the first 4 bytes to contain the *offset* into the LM to the start of the buffer (data/message) to be written from,

Push onto the stack the address of the iovec in memory,

```
global.get $intFDIovec          ;; location of iovec
```

Push onto the stack the offset in LM of the message to output

```
global.get $intOutPutArea       ;; start address of msg
```

Copy the data according to the values stored in the last two entries of the stack,

i32 Memory offset location to copy the data to.

i32 The data to be copied into memory.

```
i32.store                       ;;
```

Now, for the second 4 bytes of the iovec: the length of the buffer (data/message),

```
global.get $intFDIovec      ;; location of iovec
i32.const 4                 ;; next 4 bytes
i32.add
;; calculate total length of msg and push it
;;
local.get $intResLen
global.get $intCharMsgLen
i32.add
i32.const 1                 ;; inclusive
i32.add
i32.store
```

We are now ready to build the parameter list on the stack and call the fd_write function defined at the top of our wat source code,

```
global.get $intFDDevice         ;; device type: stdout
```

```
global.get $intFDIovec          ;; address of iovec
global.get $intFDIovecs         ;; number of iovecs
global.get $intFDBytesWritten   ;; location to store number of bytes output
call $wasi_fd_write
```

Now, we proceed to discuss the *bin2dec* function.

We convert a binary value to decimal digit equivalents using the division by 10 method, and gathering the remainders, in reverse order, as the digits, adding 48 decimal to equate to their corresponding ascii characters, e.g.

Given the number 38 in binary, 00100110,
we divide by 10 decimal to get, 3 with 8 remaining. The eight is added to 48 to get 56, which is the character ′8′ in ascii. Because the division is integer, the result is 3 in binary 0000011. We test the result and if it is not zero then we loop back to repeat the above, dividing the result by 10 decimal we get 0 plus 3 remaining. The remainder is added to 48 decimal to get 51 decimal which corresponds to the ascii character ′3′, which is prepended to the previous ′8′ character. The result of the division now leaves us with 00000000 binary as the result. We test for a zero value and terminate the loop.

The remainder from a division is derived from the *i32* method, *i32.rem_u*, which pops two *i32* values off the stack, divides the first by the second, takes the remaining amount, and then pushes it onto the stack. This looks at the values as unsigned integers (positive values only).

The function returns the address to the start of the ascii decimal string.

Some notes on new syntax encountered within the *bin2dec* function.

Conditional type test,

i32.eqz

Pops the value stored at the stack and evaluates it. If it is zero, 1 is pushed onto the stack, otherwise, 0 is pushed onto the stack. There are several conditional expressions that each type can process, e.g.,

- ne, not equal to, compares two values from the stack;
- gt_s, check if the first is greater than the second on the stack: signed, i.e., tested algebraicly;
- gt_u, same as gt_s but usnsigned: logically tested;
- etc., see the reference manual at https://coderundebug.com/learn/wat-reference/ under i32/condition for more tests.

Each type, *i32*, *i64*, *f32*, *f64*, has their particular set of condtions

Conditional blocks,

if/end

The *if* instruction will pop the last item off the stack and see if it is 0 or not. If the condition value is 0 then the code inside the if block is not executed. If the condition value is something other than 0 then it will continue to execute whatever is inside the block.

There is also an *else* version, that follows the standard logic,

```
if
    . . .
else
    . . .
end
```

One can return from a function anytime through the standard *return* instruction,

```
(return (local.get $ptr))
```

This is an s-expression that first pushes the value stored at *$ptr* onto the stack then returns to the caller.

The general format for looping is,

```
loop    $the_name_for_the_start_of_loop

    . . .

end
```

This will process the body of the loop exactly once! In order to continue looping you must supply a branch back instruction within the body of the loop. In our example, this is accomplished with the following set of instructions,

```
(br_if $digit_loop (i32.gt_u (local.get $num) (i32.const 0)))
```

The *'br_if $digit_loop'* instruction pops an *i32* value off the stack and checks its value. If it is not zero then it will branch to the label (top of the loop).